

Recurrent Neural Networks

Sequential Data, Unrolling, Shared Weights, BPTT, and Gradient Problems

CSE427: Machine Learning

Learning Objectives

After this lecture, students should be able to:

- explain why ordinary feedforward networks are not ideal for sequential data;
- explain the feedback loop in Recurrent Neural Networks;
- unroll an RNN through time;
- explain why the same weights are shared across time steps;
- compute simple RNN outputs step by step;
- explain Backpropagation Through Time;
- explain vanishing and exploding gradient problems.

RNN = neural network for ordered/sequential data

Why Do We Need RNNs?

Many real-world datasets are sequential.

stock prices over days, words in a sentence, speech over time

In sequential data, order matters.

“not good” $\neq$ “good not”

A normal feedforward network expects a fixed-size input.

But sequence lengths may differ:

Company A: 9 days of stock data

Company B: 5 days of stock data

RNNs can process variable-length sequential data.

Main Difference: Feedback Loop

A feedforward network sends information only forward.

$$x \rightarrow h \rightarrow y$$

An RNN has a feedback loop.

The previous hidden state is reused at the next time step.

At time t , the RNN uses:

$$x_t = \text{current input}, \quad h_{t-1} = \text{previous hidden state}$$

and computes:

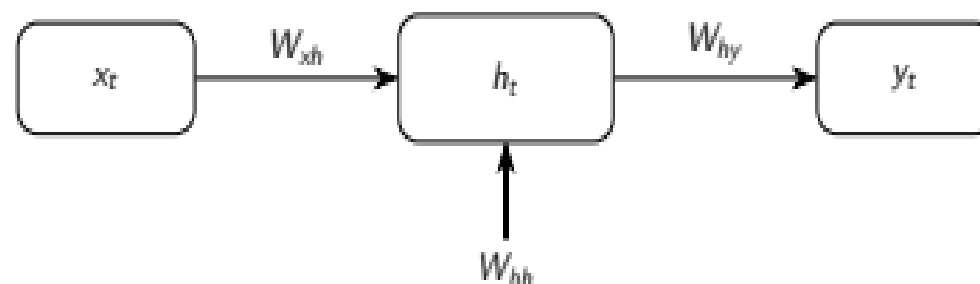
$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Then output:

$$y_t = g(W_{hy}h_t + b_y)$$

h_t acts like memory.

RNN Feedback Loop Diagram



The feedback loop means:

h_t depends on h_{t-1}

So the model remembers previous information.

This is why RNNs are useful for time series and text.

What Does Unrolling Mean?

The feedback loop is hard to follow directly.
So we draw one copy of the RNN for each time step.
This is called:

unrolling the RNN

For a sequence:

x_1, x_2, x_3

the RNN becomes:

$$x_1 \rightarrow h_1 \rightarrow y_1$$

$$x_2 \rightarrow h_2 \rightarrow y_2$$

$$x_3 \rightarrow h_3 \rightarrow y_3$$

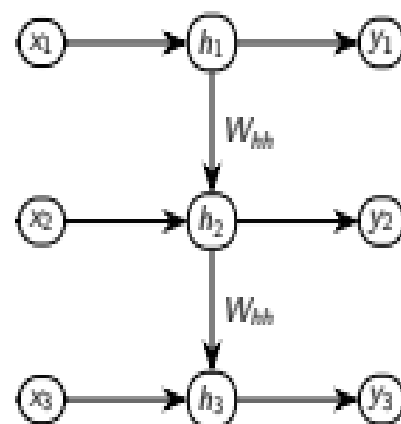
with:

$$h_1 = f(W_{xh}x_1 + W_{hh}h_0 + b_h)$$

$$h_2 = f(W_{xh}x_2 + W_{hh}h_1 + b_h)$$

$$h_3 = f(W_{xh}x_3 + W_{hh}h_2 + b_h)$$

Unrolled RNN Diagram



Unrolling converts the loop into a chain across time.

The same RNN cell is reused at each time step.

Shared Weights Across Time

When we unroll an RNN, each copy shares the same weights and biases.

$$h_1 = f(W_{xh}x_1 + W_{hh}h_0 + b_h)$$

$$h_2 = f(W_{xh}x_2 + W_{hh}h_1 + b_h)$$

$$h_3 = f(W_{xh}x_3 + W_{hh}h_2 + b_h)$$

The same parameters are reused:

$$W_{xh}, \quad W_{hh}, \quad b_h$$

So:

Longer sequence does not mean more trainable parameters.

If we have 5 inputs, unroll 5 times.

If we have 9 inputs, unroll 9 times.

But the trainable weights remain the same.

Stock Price Prediction

Assume stock prices are scaled between 0 and 1.

0 = Low, 0.5 = Medium, 1 = High

Goal:

previous prices $\rightarrow$ tomorrow's price

Example patterns:

$(0, 0) \rightarrow 0$

Low for two days, likely low tomorrow.

$(0, 0.5) \rightarrow 1$

Low then medium, likely higher tomorrow.

$(1, 0.5) \rightarrow 0$

High then medium, likely lower tomorrow.

$(1, 1) \rightarrow 1$

High for two days, likely high tomorrow.

A Very Simple Scalar RNN

Use this simple RNN:

$$h_t = \text{ReLU}(W_1 x_t + W_2 h_{t-1} + b_h)$$

$$\hat{y}_t = W_o h_t + b_o$$

Let:

$$W_1 = 1.8, \quad W_2 = -0.5, \quad W_o = 1.1$$

$$b_h = 0, \quad b_o = 0$$

Meaning:

W_1 = input weight

W_2 = feedback/recurrent weight

W_o = output weight

The recurrent weight controls influence from previous time.

Example 1: Low, Low

Input sequence:

$$x_1 = 0, \quad x_2 = 0, \quad h_0 = 0$$

Step 1:

$$h_1 = \text{ReLU}(1.8x_1 - 0.5h_0) = \text{ReLU}(1.8(0) - 0.5(0)) = 0$$

Step 2:

$$h_2 = \text{ReLU}(1.8x_2 - 0.5h_1) = \text{ReLU}(1.8(0) - 0.5(0)) = 0$$

Output:

$$\hat{y} = 1.1h_2 = 1.1(0) = 0$$

$\hat{y} = 0$, so predicted tomorrow price is Low.

Example 2: Low, Medium

Input sequence:

$$x_1 = 0, \quad x_2 = 0.5, \quad h_0 = 0$$

Step 1:

$$h_1 = \text{ReLU}(1.8(0) - 0.5(0)) = 0$$

Step 2:

$$h_2 = \text{ReLU}(1.8(0.5) - 0.5h_1)$$

$$h_2 = \text{ReLU}(0.9 - 0) = 0.9$$

Output:

$$\hat{y} = 1.1h_2 = 1.1(0.9) = 0.99$$

$\hat{y} \approx 1$, so predicted tomorrow price is High.
--

Example 3: High, Medium

Input sequence:

$$x_1 = 1, \quad x_2 = 0.5, \quad h_0 = 0$$

Step 1:

$$h_1 = \text{ReLU}(1.8(1) - 0.5(0)) = 1.8$$

Step 2:

$$h_2 = \text{ReLU}(1.8(0.5) - 0.5h_1)$$

$$h_2 = \text{ReLU}(0.9 - 0.5(1.8))$$

$$h_2 = \text{ReLU}(0.9 - 0.9) = 0$$

Output:

$$\hat{y} = 1.1h_2 = 0$$

$\hat{y} = 0$, so predicted tomorrow price is Low.

Example 4: High, High

Input sequence:

$$x_1 = 1, \quad x_2 = 1, \quad h_0 = 0$$

Step 1:

$$h_1 = \text{ReLU}(1.8(1) - 0.5(0)) = 1.8$$

Step 2:

$$h_2 = \text{ReLU}(1.8(1) - 0.5h_1)$$

$$h_2 = \text{ReLU}(1.8 - 0.5(1.8))$$

$$h_2 = \text{ReLU}(1.8 - 0.9) = 0.9$$

Output:

$$\hat{y} = 1.1h_2 = 1.1(0.9) = 0.99$$

$\hat{y} \approx 1$, so predicted tomorrow price is High.
--

Using More Than Two Days

For two days:

$$x_1, x_2$$

we unroll the RNN twice.

For three days:

$$x_1, x_2, x_3$$

we unroll the RNN three times.

For T days:

$$x_1, x_2, \dots, x_T$$

we unroll the RNN T times.

Number of unrolled copies depends on sequence length.

But:

The same weights are shared across all copies.

So the number of trainable parameters does not increase with T .

Many-to-One RNN

Many-to-one means:

many inputs $\rightarrow$ one final output

Examples:

stock prices over many days $\rightarrow$ tomorrow's price

sentence $\rightarrow$ sentiment

patient records over time $\rightarrow$ risk prediction

Usually we use the final hidden state:

$$h_T$$

as a summary of the sequence.

$$\hat{y} = \text{softmax}(Wh_T + b)$$

h_T stores information from the sequence.

Many-to-Many RNN

Many-to-many means:

many inputs $\rightarrow$ many outputs

Example: part-of-speech tagging.

Janet will back the bill

NNP MD VB DT NN

At each time step:

$$y_t = \text{softmax}(W_{hy}h_t + b_y)$$

So:

x_t = word at time t

h_t = hidden state at time t

y_t = tag prediction at time t

One output is produced for each input.

Autoregressive Generation

In language modeling, an RNN can generate text one token at a time.

Example:

$\langle s \rangle \rightarrow \text{So}$

$\text{So} \rightarrow \text{long}$

$\text{long} \rightarrow \text{and}$

At each step:

$$P(x_{t+1} | x_1, x_2, \dots, x_t)$$

is predicted.

Then the predicted token can be fed back as the next input.

Autoregressive generation predicts the next item from previous items.

Training an RNN

To train an RNN, we compare predicted values with observed values.

For regression, one possible loss is Sum of Squared Residuals:

$$SSR = \sum_{i=1}^n (Observed_i - Predicted_i)^2$$

For classification, we usually use cross-entropy loss.

To update weights, we calculate gradients:

$$\frac{\partial L}{\partial W_{xh}}, \quad \frac{\partial L}{\partial W_{hh}}, \quad \frac{\partial L}{\partial W_{hy}}$$

Because the same weights are reused across time, the gradient receives contributions from multiple time steps.

Backpropagation in RNNs is called Backpropagation Through Time.

BPTT: One Input Case

Suppose we use only today's value to predict tomorrow.

$$\text{Predicted} = \text{Today} \times W_1$$

Loss:

$$SSR = \sum_{i=1}^n (\text{Observed}_i - \text{Predicted}_i)^2$$

By chain rule:

$$\frac{dSSR}{dW_1} = \frac{dSSR}{d\text{Predicted}} \cdot \frac{d\text{Predicted}}{dW_1}$$

Now:

$$\frac{dSSR}{d\text{Predicted}} = \sum_{i=1}^n -2(\text{Observed}_i - \text{Predicted}_i)$$

and:

$$\frac{d\text{Predicted}}{dW_1} = \text{Today}$$

$$\frac{dSSR}{dW_1} = \sum_{i=1}^n -2(\text{Observed}_i - \text{Predicted}_i)(\text{Today})$$

BPTT: Two Input Case

Suppose we use yesterday and today.

Unrolled prediction:

$$\text{Predicted} = (\text{Yesterday} \times W_1 \times W_2) + (\text{Today} \times W_1)$$

Differentiate with respect to W_1 :

$$\frac{d\text{Predicted}}{dW_1} = (\text{Yesterday} \times W_2) + \text{Today}$$

Therefore:

$$\frac{dSSR}{dW_1} = \frac{dSSR}{d\text{Predicted}} \cdot \frac{d\text{Predicted}}{dW_1}$$

$$\frac{dSSR}{dW_1} = \sum_{i=1}^n -2(\text{Observed}_i - \text{Predicted}_i)[(\text{Yesterday} \times W_2) + \text{Today}]$$

The older input is multiplied by W_2 .

BPTT: Three Input Case

Let:

$$D_{-2} = \text{day before yesterday}, \quad D_{-1} = \text{yesterday}, \quad D_0 = \text{today}$$

Prediction:

$$\text{Predicted} = (D_{-2} W_1 W_2^2) + (D_{-1} W_1 W_2) + (D_0 W_1)$$

Differentiate with respect to W_1 :

$$\frac{d\text{Predicted}}{dW_1} = D_{-2} W_2^2 + D_{-1} W_2 + D_0$$

Thus:

$$\frac{dSSR}{dW_1} = \sum_{i=1}^n -2(\text{Observed}_i - \text{Predicted}_i)[D_{-2} W_2^2 + D_{-1} W_2 + D_0]$$

Older inputs get multiplied by powers of W_2 .

General Pattern in BPTT

For a longer sequence:

$$D_{-k}, \dots, D_{-2}, D_{-1}, D_0$$

older terms contain higher powers of the recurrent weight.

$$D_{-1} \rightarrow D_{-1} W_2$$

$$D_{-2} \rightarrow D_{-2} W_2^2$$

$$D_{-3} \rightarrow D_{-3} W_2^3$$

$$D_{-20} \rightarrow D_{-20} W_2^{20}$$

So the gradient depends strongly on powers of W_2 .

This causes vanishing or exploding gradients.

Why Gradient Problems Happen

Older inputs are repeatedly multiplied by the recurrent weight.

If:

$$|W_2| < 1$$

then:

$$W_2^k \rightarrow 0$$

as k becomes large.

This causes:

vanishing gradient

If:

$$|W_2| > 1$$

then:

$$W_2^k \rightarrow \text{very large}$$

This causes:

exploding gradient

Vanishing Gradient Example

Suppose:

$$W_2 = 0.5$$

Then:

$$0.5^1 = 0.5, \quad 0.5^2 = 0.25, \quad 0.5^3 = 0.125$$

$$0.5^{10} = 0.00098 \approx 0.001$$

$$0.5^{20} = 0.00000095 \approx 0.000001$$

So an old input like D_{-20} becomes almost irrelevant:

$$D_{-20} \times 0.5^{20} \approx 0$$

Old information contributes almost nothing to the gradient.

This is the vanishing gradient problem.

Exploding Gradient Example

Suppose:

$$W_2 = 2$$

Then:

$$2^1 = 2, \quad 2^2 = 4, \quad 2^3 = 8$$

$$2^{10} = 1024$$

$$2^{20} = 1,048,576$$

So an old input like D_{-20} can dominate the gradient:

$$D_{-20} \times 2^{20}$$

Old information becomes too large and destabilizes training.

This is the exploding gradient problem.

Gradient Problem Summary

Condition	Mathematical Effect	Problem
$ W_2 < 1$	$W_2^k \rightarrow 0$	Vanishing gradient
$ W_2 > 1$	$W_2^k \rightarrow \infty$	Exploding gradient
$ W_2 \approx 1$	More stable	Better gradient flow

Common solutions:

- LSTM
- GRU
- gradient clipping
- careful initialization

LSTM and GRU are improved RNN architectures.

Final Summary

- RNNs are designed for sequential data.
- RNNs use feedback loops to pass information through time.
- The hidden state acts like memory.
- Unrolling shows the RNN as a chain across time steps.
- The same weights are shared at every time step.
- RNNs can handle variable-length sequences.
- Training RNNs uses Backpropagation Through Time.
- Older inputs are multiplied repeatedly by recurrent weights.
- This can cause vanishing or exploding gradients.
- LSTM and GRU help reduce these problems.

RNN = neural network with memory over time

One-Line Teaching Version

A feedforward network sees one fixed input.

An RNN reads a sequence step by step.

It remembers the past using a hidden state.

The same weights are reused at every time step.

Training across time is called BPTT.

Repeated multiplication can cause vanishing or exploding gradients.